Effat University

Lab 6: Project Management

FALL 2026 – CS3151

ENGINEERING COLLEGE – COMPUTER SCIENCE DEPARTMENT

INSTRUCTORS NAME: Dr. Passent Elkafrawy Leen kasem, Reema Abualnaja, Busra Suleymanoglu, Amira Atiah		
COURSE NAME: Software Engineering	COURSE NUMBER: CS3151	SECTION: 1-2-3

Project Planning, Prioritization, Risk Management, and Simple Docker Deployment

LEARNING OUTCOMES

By the end of this lab, students will be able to:

- Identify and prioritize system requirements
- Convert requirements into a product backlog
- Apply MoSCoW prioritization
- Break requirements into development tasks
- Plan sprints and milestones
- Identify project risks and mitigation strategies
- Understand how implementation tasks relate to deployment using Docker

---

Part 1 – Backlog meeting

The team should evaluate the proposed projects in term of the following from the brief summeries,:

- scope
- complexity
- risks
- feasibility
- business value

then choose one to continue working on next parts

Below are **four project proposals written like brief meeting summaries**.

Project Proposal 1 – Smart Event Management Platform

A web platform that allows organizations to create and manage events such as conferences, workshops, and competitions. Organizers can create events, publish schedules, manage registrations, and send notifications to participants. Users can browse events, register online, and receive reminders.
The system should support basic event management features including event creation, participant registration, schedule display, and email notifications.
The goal is to launch a **minimum viable product (MVP)** that supports small events with up to 200 participants.

Effat University

Lab 6: Project Management

FALL 2026 – CS3151

Project Proposal 2 – University Internship Management System

A system designed to help universities manage student internships with external companies. Students can browse available internships, apply for positions, and track application status. Companies can post internship opportunities and review applicants. Faculty members can monitor internship progress and approve placements.
The platform should support internship posting, application submission, approval workflows, and basic reporting.
The MVP should allow **students to apply to internships and companies to review candidates**.

Project Proposal 3 – Smart Parking Finder Application

A mobile-friendly application that helps drivers find available parking spaces in busy areas such as malls or campuses. Users can view available parking spots on a map and receive directions to the nearest available location. Parking managers can update space availability through a simple interface.
The system should include parking availability tracking, location mapping, and user notifications.
The MVP should allow **users to view available parking spaces and parking managers to update availability**.

Project Proposal 4 – Campus Event and Activity Portal

A centralized portal for university clubs and departments to publish campus events, workshops, and student activities. Students can browse upcoming events, register to attend, and receive announcements.
Clubs and departments can create events, manage participant lists, and post updates.
The MVP should support **event publication, student registration, and event announcements**.

Evaluation of the four proposals (scope, complexity, risks, feasibility, business value):

Proposal	Scope	Complexity	Risks	Feasibility	Business value
Smart Event Management Platform	Medium: event creation, schedules, registration, email notifications	Medium	Medium: email delivery, schedule display, capacity limits	Good for an MVP of up to 200 participants	Good, but general-purpose and overlaps with existing tools
University Internship Management System	Large: three user types (students, companies, faculty) plus approval workflows and reports	High: multi-party workflows and approvals	Medium–High: depends on companies and faculty adopting it	Moderate: only a reduced MVP fits in 3 months	High and measurable (placements, reporting)
Smart Parking Finder	Medium: availability tracking, maps, directions, notifications	High: maps and routing, near-real-time data, mobile UI	High: stale or inaccurate availability data, map API dependency, manager discipline	Low–Moderate	Medium: limited to specific venues and their managers
Campus Event and Activity Portal	Small–Medium: publish events, register, announcements	Low: standard web CRUD, one institution, no external parties	Low: mainly adoption and scope creep	High: realistic for a 3-developer team in 6 sprints	Medium–High: immediate value to students, clubs and departments

- Which project has the **clearest business value**?

Proposal 2 (Internship Management) has the clearest and most measurable institutional value (placements, faculty oversight, reporting). Proposal 4 is a close second because its value is immediate and visible to every student and club.

- Which project appears **least complex** for a student team?

Proposal 4 (Campus Event and Activity Portal). It has one institution, no external parties, no maps or real-time data and no approval workflows, so it is standard web CRUD.

- Which project has the **highest potential risks**?

Proposal 3 (Smart Parking Finder). It depends on accurate near-real-time availability data, map and location services, and parking managers keeping the data updated, and all of these can fail.

- Which project can realistically deliver an MVP in **3 months**?

Proposal 4. Its MVP (publish events, register, announcements) is small and well defined, which fits 6 two-week sprints for a team of 3 developers. Proposal 1 could also fit, but Proposal 4 is simpler.

- Which project would you recommend starting first and why?

Proposal 4. It is the lowest-risk option with a clear MVP and quick user feedback, and its pieces (users, events, registration, notifications) can later grow into larger systems such as Proposal 1.

Selected project: Campus Event and Activity Portal - Project Proposal 4
Justification: We chose Proposal 4 because it has the lowest complexity and risk while still giving clear value to students, clubs and departments. Its MVP (event publication, student registration, event announcements) is small and well defined, so a 3-developer team can deliver it within 3 months. Feedback from a real pilot group is also easy to get on campus.

Consider the rest examples based on this Case Study

Campus Event and Activity Portal: The university wants a centralized web portal where clubs and departments publish campus events, workshops and student activities. Students browse upcoming events, register to attend and receive announcements.
Users: (1) Club/department organizers, (2) Students (participants), (3) System administrators (manage organizer accounts and moderate content).
Goal: Launch a Minimum Viable Product (MVP) within 3 months.

Part 2 – Requirement Identification

1. List **five** functional requirements for the system.

Example format:

Requirement ID	Requirement Description
FR1	The system shall allow an authorized club or department organizer to create, edit and publish an event containing a title, date, time, location, description and registration capacity.
FR2	The system shall allow students to browse and search published upcoming events (by date, category or club) and view their complete event details.
FR3	The system shall allow an authenticated student to register for an event when registration is open and capacity is available, shall prevent duplicate registration, and shall allow the student to cancel.
FR4	The system shall allow the event organizer to view the participant list and registration count for each event they manage.
FR5	The system shall allow an organizer to publish an announcement or schedule update that is visible to registered participants and is sent to them by email.

2. Identify **three** non-functional requirements.

Requirement ID	Non-Functional Requirement
NFR1	Security: only authenticated users can register for events, and only authorized organizers can create, modify or manage their own events and participant lists (role-based access; passwords hashed; HTTPS).
NFR2	Performance: event browsing, event-detail pages and registration confirmation respond within 2 seconds for at least 95% of requests with up to 200 concurrent users.
NFR3	Data integrity: a successful registration is stored exactly once, and the displayed participant count always equals the number of active registrations for the event.

Effat University

Lab 6: Project Management

FALL 2026 – CS3151

Part 3 – Product Backlog Creation

Clarification: Which comes first, the user story or the requirements?

In Agile, the user story usually comes first. A story is a short, deliberately incomplete statement of a user need, often described as "a placeholder for a conversation." The detailed requirements are then discovered from the story through discussion with the product owner and stakeholders, and are recorded as **acceptance criteria**.

This is known as the 3 Cs: **Card** (the written story), **Conversation** (the team discusses the details), and **Confirmation** (acceptance criteria that make the story testable).

Typical flow: Business need / product vision → Epics → User stories → Acceptance criteria (detailed requirements)

Stories do not come from nowhere: they are derived from the product vision, business goals, and user research. When a formal requirements list already exists (for example in contract work or regulated industries), teams break those requirements down into user stories. This lab follows that hybrid approach: you identify requirements in Part 2, then express them as user stories here. Either way, each user story should end up with clear acceptance criteria.

Story ID	User Story
US1	As a club organizer I want to create and publish an event with title, description, date, venue and capacity So that students can discover it and register.
US2	As a student I want to browse and search upcoming events by date, category or club So that I can find activities that interest me.
US3	As a student I want to register for an event So that my place is reserved and the organizer knows I will attend.
US4	As a club organizer I want to view the list of registered participants So that I can plan the event and manage attendance.
US5	As a club organizer I want to post announcements for my event So that registered students are informed about changes and updates.

Part 4 – Requirement Prioritization (MoSCoW Method)

Assign a MoSCoW priority to each user story.

Story ID	User Story	Priority
US1	As a club organizer I want to create and publish an event with title, description, date, venue and capacity.	Must Have
US2	As a student I want to browse and search upcoming events by date, category or club.	Must Have
US3	As a student I want to register for an event.	Must Have
US4	As a club organizer I want to view the list of registered participants.	Should Have
US5	As a club organizer I want to post announcements for my event.	Must Have

Rationale
US1: no events means no portal; everything else depends on it.
US2: discovery is the main value for students.
US3: registration is part of the MVP.
US4: important for organizers, but the MVP works without it (registrations are still stored).
US5: announcements are part of the stated MVP (basic posting on the event page). Email delivery of them is a Should Have.
Note: Won't Have (this release): QR-code check-in, paid ticketing, calendar sync, a native mobile app, and an analytics dashboard. Could Have: event images, favourite events, and a CSV export of the participant list.

Questions

1. Which feature is **most critical for the MVP**?
US1 – create and publish an event. Nothing else can work without events; US3 (registration) is the core value built right on top of it.
2. Which feature can be postponed to a later release?
US4 (participant list view and export) can be postponed. The MVP still works without it, because registrations are stored. Email delivery of announcements, event images and a CSV export can also wait.
3. What could happen if priorities are not clearly defined?
The team may spend time on nice-to-have features and miss the MVP deadline. It also leads to scope creep, disagreement inside the team about what to build, wasted effort, and a product that does not meet stakeholder needs.

Part 5 – Task Breakdown (Work Breakdown Structure)

Feature: User story chosen: US3 – As a student, I want to register for an event, so that my place is reserved.

Task ID	Task Description
T1	Design the database table for registrations (student ID, event ID, timestamp) with a unique constraint to prevent duplicates.
T2	Design the UI: Register / Cancel buttons on the event page and the confirmation message.
T3	Implement the registration endpoint with validation (user logged in, event is upcoming, not already registered, event not full).
T4	Store the registration and update the remaining seat count.
T5	Send a confirmation email / in-app message to the student.
T6	Implement cancellation of a registration and free the seat.
T7	Write unit and integration tests for registration and package the service in Docker.

Questions

1. Why is breaking features into tasks important for project management?
Small tasks are easier to estimate, assign and track. They show dependencies and risks early, give visible progress (done / not done), let several developers work at the same time, and are easier to test and review than one large feature.

2. Which tasks can be done in parallel?
T1 (database design) and T2 (UI design) can be done in parallel, and T5 (email template) can be started once the API contract is agreed. T3 depends on T1, T4 depends on T3, T6 depends on T4, and T7 (testing) comes last.

Part 6 – Sprint Planning

Sprint	Selected User Stories
Sprint 1	US1 (create and publish an event) and US2 (browse and search events), plus project setup: repository, database schema, login, and the Docker skeleton. Sprint goal: organizers can publish events and students can see them.
Sprint 2	US3 (register for an event), US5 (announcements) and US4 (participant list, stretch goal; first to move to the next sprint if capacity is short). Sprint goal: the full MVP flow works end to end.

Questions

1. Why should teams implement high-priority features first?
High-priority features deliver the most value and carry the most risk, so building them first gives a working product early. The team gets feedback sooner, and if time runs out the items left out are the least important.
2. What risks arise if too many tasks are added to a sprint?
The team overcommits and misses the sprint goal, work carries over, quality drops (more bugs, skipped tests), people burn out, and velocity estimates become unreliable. Stakeholders also lose trust in the plan.

Part 7 – Risk Identification

Risk ID	Risk Description	Impact	Mitigation
R1	Two or more students attempt to register for the final available spot at the same time.	Event capacity may be exceeded and the participant count may become inaccurate.	The system checks the remaining capacity before confirming each registration and rejects registration once the event is full.
R2	An unauthorized user attempts to modify an event or access its participant list.	Student information may be exposed or event information may be changed without permission.	Apply role-based access control and verify that the organizer manages the event before allowing access.
R3	An organizer changes the event time or location after students have already registered.	Registered students may receive outdated information and miss the event.	Save the updated event information and display an announcement to registered participants whenever important event details change.

Questions

1. Which risk has the highest impact?

R1 and R2 both have a high impact. R2 (unauthorized access) is the highest, because participant information and event-management functions must never be exposed to unauthorized users.

2. Which risk is most likely to occur?

R3 (scope creep) is the most likely, because many clubs and departments will request additional features during the project. R1 is the most likely technical risk when a popular event opens registration.

Part 8 – Project Milestones

Milestone	Description
M1 – Requirements and Backlog Completed	Functional and non-functional requirements, user stories, and backlog priorities are finalized.
M2 – Core Event Flow Completed	Organizers can publish events, students can browse event details, and students can register for available events.
M3 – Management and Testing Completed	Participant management and event updates are integrated, and registration, capacity, authorization, and core workflow tests are completed.
M4 – MVP Deployment Ready	The final MVP is tested and the selected Campus Event Portal service is successfully built and executed using Docker.

Effat University

Lab 6: Project Management

FALL 2026 – CS3151

Part 9 – Simple Docker Implementation

Step 1: Create the Python Application

Create a project folder and save the following code in it as `app.py`. Python uses indentation to define code blocks, so keep the indented lines exactly as shown.

`app.py`

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return """
    <div style="font-family: Arial; max-width: 700px; margin: 60px auto;">
        <h1>Campus Event & Activity Portal</h1>
        <h3>Connecting students with campus life.</h3>

        <hr>

        <h2>Event Registration Module</h2>
        <p>
            Students can discover upcoming campus events,
            view event details, and reserve their spot.
        </p>

        <p><strong>Current Feature:</strong> Event Registration</p>
        <p><strong>Status:</strong> Service is running successfully.</p>
    </div>
    """

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Step 2: Create requirements.txt

In the same folder, create `requirements.txt` listing the Python packages the application needs:

`requirements.txt`

```
flask
```

Step 3: Create the Dockerfile

In the same folder, create a file named `Dockerfile` (no file extension):

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

Each line is one build step: start from a small Python base image, set the working folder, install the dependencies, copy in the code, document the port the app listens on, and define the command that starts the app. The dependencies are copied and installed before the code so that Docker can reuse that step when only the code changes.

Step 4: Build and Run

Open a terminal in the project folder and run:

```
docker build -t campus-event-service .
docker run -p 5000:5000 campus-event-service
```

Expected Result

Docker Run:
The Docker container was executed on port 5000. The Campus Event and Activity Portal service was then accessed through localhost:5000.

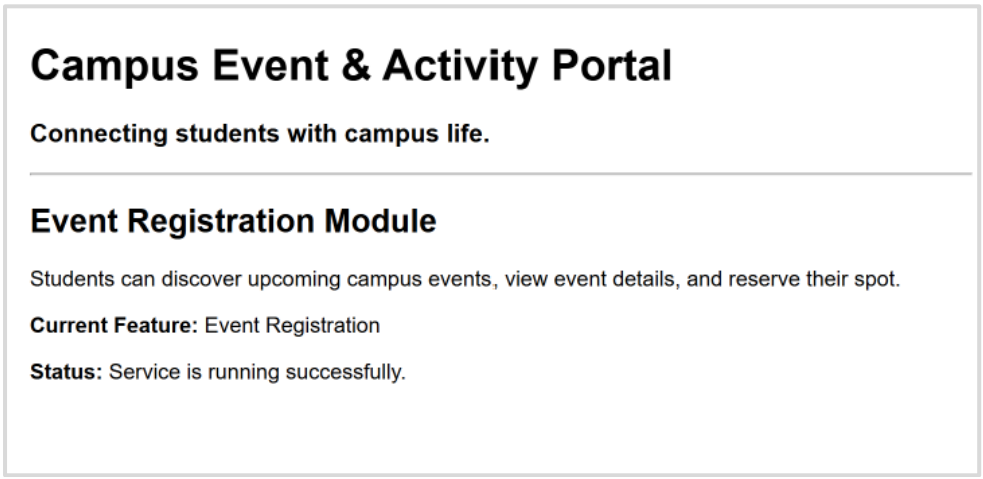


Figure 1 Event Registration module of the Campus Event and Activity Portal running successfully inside the Docker container.

Reflection Questions

1. Why is prioritization critical in software projects?

Prioritization protects the MVP from scope creep. For this portal, publishing, discovering and registering for events must be completed before effort is spent on enhancements such as participant-list export or event images. Time, people and budget are limited, so the most valuable work has to come first.

2. How does backlog management help project managers?

It turns the project scope into visible, prioritized work. The manager can see which stories form the main event workflow, which depend on others, and which can move to a later sprint if time becomes limited; it also supports sprint planning and progress tracking.

3. How can poor planning affect system architecture later?

Poor planning can create tightly coupled components that are hard to change. For example, if registration capacity is implemented separately from the participant list, concurrent registrations can produce inconsistent counts, and adding roles and permissions later forces expensive rework. Defining these relationships early reduces redesign.

4. Which part of this lab reflects real industry practice the most?

Backlog prioritization and sprint planning, because development teams constantly decide what delivers the most value within limited time and capacity instead of building every requested feature. Packaging the service in Docker also reflects how applications are deployed today.